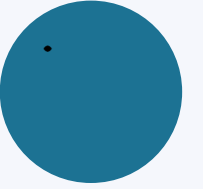


مدیریت تغییر در پلتفرم آموزش آنلاین مدارس

افزودن قابلیت نظارت هوشمند بر آزمون آنلاین (Online Exam Proctoring)

پروژه درس مهندسی نرم افزار – فاز تحول و مدیریت تغییر



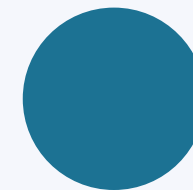
نیازمندی جدید: نظارت هوشمند بر آزمون

- ثبت خودکار رویدادهای مشکوک در حین آزمون: خروج از صفحه، تغییر مکرر تب، قطع ارتباط و عدم فعالیت طولانی
- استفاده از دوربین، میکروفون و رویدادهای سیستم برای پایش رفتار دانش آموز
- ارسال اعلان به معلم جهت بررسی وضعیت آزمون در صورت مشاهده رفتار مشکوک
- این قابلیت در طراحی اولیه سامانه پیش‌بینی نشده و به‌عنوان یک درخواست تغییر مطرح شده است

در نسخه اولیه سامانه

- ایجاد سؤال و برگزاری آزمون
- دریافت پاسخ‌ها
- محاسبه و اعلام نمره
- فرض بر معتبر بودن محیط آزمون

تأثیر بر نیازمندی‌های کارکردی



فعال بودن دورین دانش‌آموز در طول آزمون

1

بررسی قطع شدن تصویر یا صدا

2

تشخیص خروج کاربر از صفحه آزمون

3

ثبت هشدار در صورت مشاهده رفتار مشکوک

4

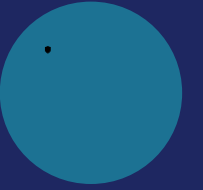
ذخیره رویدادهای مرتبط با جلسه آزمون

5

مشاهده گزارش کامل نظارت توسط مدرس و تصمیم‌گیری نهایی

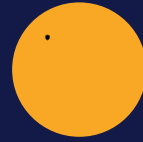
6

تأثیر بر نیازمندی‌های غیرکارکردی



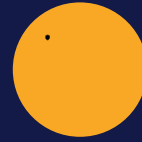
امنیت و محرمانگی

افزایش اهمیت حفاظت از داده‌های تصویری و صوتی کاربران



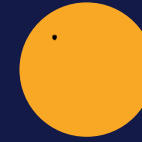
سرعت پردازش

ثبت هشدارها باید تقریباً به صورت لحظه‌ای انجام شود



حریم خصوصی

لزوم رعایت حریم خصوصی دانش‌آموزان در پایش رفتار



قابلیت اطمینان

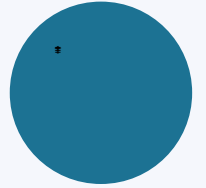
پایداری سیستم در طول برگزاری آزمون‌های حساس



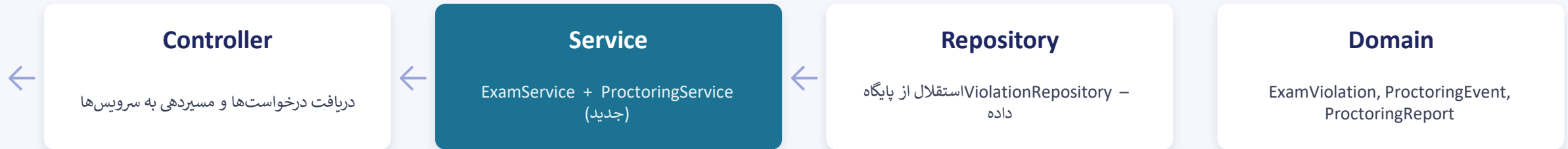
مقیاس‌پذیری

امکان پردازش هم‌زمان تعداد زیادی آزمون فعال

تأثیر تغییر بر معماری سیستم



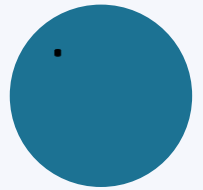
معماری لایه‌ای و سرویس‌گرا نیازی به بازطراحی اساسی ندارد؛ تنها یک سرویس مستقل به معماری اضافه می‌شود



معماری رویدادمحور — (Event-Driven Architecture) ارتباط بین سرویس‌ها بدون وابستگی مستقیم

- افزودن سرویس مستقل ProctoringService بدون تغییر در ExamService و Controller اصلی
- ارتباط با سایر سرویس‌ها از طریق API Gateway برای حفظ تمرکز و کنترل ارتباطات
- حفظ کامل اصل تفکیک مسئولیت (Single Responsibility Principle)

موجودیت‌های جدید در لایه Domain

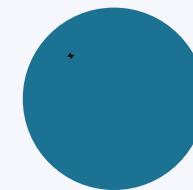


موجودیت	توضیحات فیلدها
Exam Violation	شناسه تخلف، آزمون و دانشجو - نوع تخلف - زمان رخداد - میزان اهمیت
Proctoring Event / Session	اطلاعات جلسه نظارت، رویدادهای دریافتی و وضعیت پردازش هر رویداد
Proctoring Report	گزارش نهایی تخلفات یک آزمون جهت بررسی و تصمیم‌گیری مدرس

چرا این موجودیت‌ها اهمیت دارند؟

- امکان ایجاد تاریخچه‌ای از تخلفات هر آزمون برای تحلیل رفتار دانش‌آموزان
- استفاده در آینده برای گزارش‌گیری مدیریتی و تحلیل آماری تخلفات
- نگهداری اطلاعات نظارت به‌صورت مستقل از اطلاعات اصلی آزمون

معماری رویدادمحور و رویدادهای جدید



ExamStarted



TabSwitched



CameraDisconnected



NetworkInterrupted



SuspiciousActivityDetected



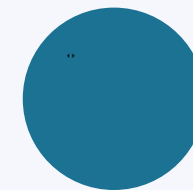
ExamViolationReported



EventPublisher → منتشر می کند رویدادها را → Notification | Logging | Proctoring | Reporting

- هر Listener مسئولیت اختصاصی خود را بدون وابستگی مستقیم به سایر اجزا انجام می دهد
- افزودن رویداد یا Listener جدید بدون تغییر در هسته سیستم امکان پذیر است — انطباق کامل با اصل باز/بسته (OCP)

تأثیر تغییر بر کد و پیاده‌سازی موجود



Domain

توسعه کلاس (Exam وضعیت نظارت و قوانین آزمون (و Submission ثبت وضعیت تخلفات احتمالی)

Service

افزودن – ProctoringService توسعه ExamService برای ایجاد و پایان جلسه نظارت

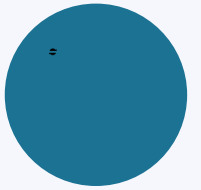
Controller

افزودن API های جدید: شروع نظارت، دریافت هشدارها، پایان جلسه، مشاهده گزارش

(UI) رابط کاربری

نمایش وضعیت دوربین/میکروفون/اتصال و دریافت مجوز دسترسی پیش از آزمون

بازاندیشی در طراحی اولیه



اگر این نیازمندی از ابتدا مشخص بود، طراحی چه تفاوتی می کرد؟

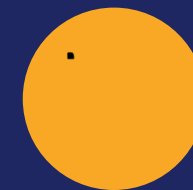
طراحی فعلی (بدون پیش بینی نیازمندی)

- ماژول آزمون فقط مسئول برگزاری، ثبت پاسخ و محاسبه نمره
- فرض بر معتبر بودن محیط برگزاری آزمون
- بدون مؤلفه مستقل برای پایش رفتار یا رویدادهای امنیتی

در صورت اطلاع از ابتدای پروژه

- زیرسامانه مستقل Proctoring / Assessment Monitoring Module
- مدل دامنه تخصصی از ابتدا : ExamViolation, ProctoringEvent, ViolationReport
- تفکیک کامل مسئولیت نظارت از ابتدا در قالب ProctoringService

ارزیابی کیفیت معماری اولیه



معماری لایه‌ای

جداسازی مناسب Domain،
Repository، Service و Controller



الگوی Repository

استقلال منطق ذخیره‌سازی تخلفات از
سایر بخش‌های سیستم



معماری Event-Driven

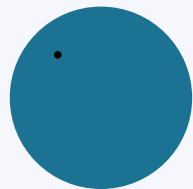
افزودن رویدادهای جدید بدون تغییر
گسترده – اصل OCP



سرویس‌های مستقل

حفظ اصل تفکیک مسئولیت (SRP) در
سراسر سیستم

نتیجه: افزودن قابلیت نظارت هوشمند بدون بازطراحی اساسی هسته سیستم امکان‌پذیر است



تخمین تلاش تغییر (Effort Estimation)

فعالیت	ساعت-نفر
تحلیل نیازمندی جدید	4
بازطراحی مدل دامنه	5
توسعه لایه Repository	3
توسعه ProctoringService	9
اصلاح Eventها	3
اصلاح NotificationService	3
اصلاح AppController	4
تست واحد	4
تست یکپارچگی	4
بروزرسانی مستندات	3

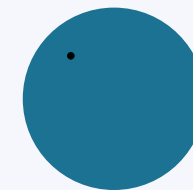
42

مجموع نفر-ساعت

سطح پیچیدگی: متوسط

تلاش منطقی برای قابلیت که چندین لایه سیستم را تحت تاثیر قرار می دهد اما نیاز به بازنویسی ندارد.

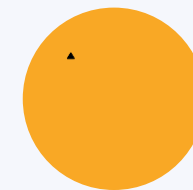
تخمین زمان اجرای تغییر (Schedule)



روز 0.5	تحلیل نیازمندی
روز 1	طراحی و بازطراحی
روز 1	توسعه Repository و Domain
روز 1.5	توسعه Event و Service
روز 0.5	توسعه Controller و Notification
روز 1	اجرای تست‌ها
روز 0.5	بروزرسانی مستندات

مدت زمان کل: حدود 5 تا 6 روز (تا 1 روز افزایش در صورت بروز اشکالات پیش‌بینی نشده)

تحلیل ریسک



ریسک	احتمال	اثر	اولویت
تداخل با منطق فعلی ExamService	متوسط	زیاد	بالا
ثبت اشتباه تخلفات	متوسط	متوسط	متوسط
افزایش پیچیدگی معماری	کم	متوسط	پایین
عدم ارسال صحیح اعلان‌ها	کم	متوسط	پایین
افزایش زمان تست	متوسط	کم	پایین

مهم‌ترین راهکارهای کاهش ریسک

- جداسازی کامل منطق نظارت در ProctoringService و اجرای تست‌های یکپارچگی
- تعریف قوانین دقیق برای ثبت رویدادها و اعتبارسنجی داده‌ها
- طراحی Test Case های خودکار برای سناریوهای اصلی نظارت

جمع‌بندی نهایی

۴۲

نفر-ساعت تلاش

۵ تا ۶

روز کاری

متوسط

سطح پیچیدگی

- معماری لایه‌ای و الگوهای طراحی مناسب (Repository, Event-Driven, Strategy) امکان افزودن قابلیت نظارت هوشمند را بدون بازنویسی هسته سیستم فراهم کردند
- افزودن ProctoringService به عنوان سرویسی مستقل، اصول SRP و OCP را در سراسر سیستم حفظ کرده است
- تصمیمات معماری اتخاذ شده در مراحل اولیه پروژه، پاسخگویی مناسبی به تغییرات آینده را ممکن ساخته‌اند